

AI RESEARCH ENGINEER

90-Day Learning, Research & Paper Roadmap

AI / ML → Research Engineering → AI + Quantum Communication + QKD + 6G

Profile track: E&TC Engineering → AI/ML Engineer → Research Engineer → Quantum Communication + AI Researcher

90-day, week-by-week roadmap synchronized with a curated paper-reading list

Quality-over-quantity paper selection with tiering, dependency graph, and reading passes

Five progressively harder implementation projects, culminating in an AI+QKD security study

Research methodology: reproduction, ablation, error analysis, and research-gap identification

Internship, Master's, and professor-outreach preparation

"Optimize for depth of understanding, not for the number of papers finished."

Personal Study Plan | Research Paper Reading Guide | Implementation Plan | Research Preparation Roadmap

TABLE OF CONTENTS

1. Executive Summary
2. My 90-Day Objective
3. AI Research Engineer Competency Map
4. Complete AI Knowledge Tree
5. Mathematics Prerequisite Roadmap
6. Classical ML Research Foundation
7. Deep Learning Research Foundation
8. NLP Research Evolution
9. ResNet Research Journey
10. Transformer Research Journey
11. BERT → GPT → LLM Evolution
12. Information Retrieval → RAG Evolution
13. Modern LLM Research Landscape
14. Research Methodology
15. Paper Reading Methodology
16. Complete 12-Week Roadmap
17. Paper Dependency Graph
18. Deep-Read / Implementation / Reproduction Paper Lists
19. AI + Quantum / QKD / 6G Research Roadmap
20. Research Gap Identification Framework
21. Project Roadmap
22. Internship Preparation
23. Master's Preparation
24. Professor / Research Outreach Preparation
25. Research Notebook Template
26. Weekly Research Schedule
27. 90-Day Checklist
28. Papers to Skip for Now
29. Next Steps After Day 90

01 Executive Summary

This document is a 90-day plan for turning your E&TC background and existing QKD/BB84 final-year project into a genuine AI research engineering trajectory. It treats research-paper reading as a means to an end — understanding deep enough to implement, reproduce, critique, and extend — never as a quota to complete. Every paper included has a stated reason for being there; papers that would not change your understanding or your goals are explicitly left out, and named in the 'Papers to Skip for Now' section.

The plan runs in three layers that move together every week: **engineering** (Python → FastAPI → PostgreSQL → React/Next.js → Docker → deployment), **AI/ML research foundations** (classical ML → deep learning → CNNs/ResNet → Transformers → BERT/GPT → RAG → modern LLM techniques), and **specialization** (AI applied to Quantum Key Distribution, quantum communication, physical-layer security, and 6G). The specialization layer is built to connect directly to your existing BB84 project rather than replace it.

Five projects anchor the 90 days — an AI Resume Analyzer, a ResNet implementation with an ablation study, a Transformer built from scratch, an AI Research Assistant (RAG over papers), and an ML-based security analysis of BB84/QKD. Together they form a coherent GitHub portfolio and a credible research narrative for internships, Master's applications, and professor outreach.

02 My 90-Day Objective

By the end of Day 90, you should be able to do the following — this is the actual target the whole roadmap is designed against, not a list of topics covered.

Engineering

- Write clean, typed Python; use NumPy/Pandas fluently; work with Git/GitHub professionally.
- Build and document a REST API in FastAPI with validation, auth, and async endpoints.
- Design a normalized PostgreSQL schema and write non-trivial SQL (joins, indexes, CRUD).
- Build a usable React/Next.js frontend for an AI-backed application.
- Integrate an LLM API, build a RAG pipeline with a vector database, and Dockerize the result.

AI / ML

- Explain classical ML models, the math behind gradient descent, and standard evaluation metrics.
- Derive backpropagation and implement a training loop in PyTorch from first principles.
- Explain CNNs, the degradation problem, and why residual connections solve it.
- Derive scaled dot-product and multi-head attention and implement a Transformer from scratch.
- Explain the BERT/GPT split, pretraining, instruction tuning, RLHF/DPO, LoRA/QLoRA, and hallucination.

Research

- Read a paper in four passes (orientation → technical → critical → implementation) instead of linearly front-to-back.
- Reproduce a chosen experiment end-to-end, including an ablation and an error analysis.
- Critically evaluate whether a claimed quantum or AI advantage is real, and where a paper's assumptions or baselines are weak.
- Convert a limitation you find in the literature into a testable research question and hypothesis.

Specialization

Gradually specialize toward **AI + Quantum Computing + Quantum Communication + QKD + Cybersecurity + 6G**, using your BB84 project as the anchor rather than starting a new direction from zero.

03 AI Research Engineer Competency Map

Layer	Weeks	You can currently do...	You will be able to do by Day 90
Engineering	1–4, 9	E&TC; coursework; general programming exposure.	Ship a Dockerized FastAPI + PostgreSQL + React app that calls an LLM and a vector DB.
Math for ML	2, 5, 7	Engineering-level linear algebra/calculus/probability.	Derive gradient descent, backprop, and scaled dot-product attention from scratch.
Classical ML	2	Conceptual awareness only.	Train, evaluate, and compare regression/tree/SVM/clustering models with correct metrics.
Deep Learning	5–8	None / theoretical.	Implement and train CNNs, ResNet, and a Transformer in PyTorch; explain why each design works.
NLP / LLMs	4, 8, 10	User-level experience with chat LLMs.	Explain the embeddings→attention→BERT/GPT→RLHF→LoRA lineage and fine-tune a small model.
Information Retrieval / RAG	9	None.	Build a working RAG pipeline with chunking, embeddings, retrieval, reranking, and citation.
Research Methodology	6, 7, 11	Final-year-project-level exposure.	Reproduce a paper's experiment, run an ablation, and write a limitations/error analysis.
AI + Quantum/QKD/6G	12 (+ ongoing)	Strong QKD/BB84 domain knowledge from your final-year project.	Critically evaluate AI+QKD papers and run your own ML-based BB84 security-analysis study.

04 Complete AI Knowledge Tree

This is the map the whole roadmap walks through. Branches are ordered by dependency, not by importance — a branch lower down often depends on everything above it.

■ Mathematics

- Linear Algebra (vectors, matrices, eigenvalues, projections)
- Calculus (derivatives, partial derivatives, gradients, chain rule)
- Probability (distributions, expectation, Bayes' theorem)
- Statistics (variance, covariance, hypothesis testing)
- Optimization (loss functions, gradient descent, convexity)

■ Classical Machine Learning

- Regression & Classification
- Trees & Random Forest
- SVM
- Clustering (K-means) & PCA
- Evaluation (precision/recall/F1/ROC-AUC)

■ Deep Learning

- Neural networks & backpropagation
- Optimization (SGD, Adam) & regularization
- CNNs → ResNet
- Representation learning

■ NLP

- Word embeddings (Word2Vec)
- Seq2Seq
- Attention (Bahdanau)
- Transformers
- BERT
- GPT
- Modern LLMs

■ Information Retrieval

- TF-IDF
- BM25
- Dense retrieval
- DPR
- REALM
- Reranking

■ Generative AI

- LLMs & prompting
- Fine-tuning (LoRA, QLoRA)
- Quantization
- RLHF / DPO
- Evaluation & hallucination

- Agents & tool use

■ Research Methodology

- Literature review
- Baselines & ablation
- Reproducibility
- Error analysis
- Research-gap identification
- Hypothesis formation

■ AI + Quantum

- Quantum computing basics
- Quantum Machine Learning (QML)
- QKD / BB84
- Quantum communication (free-space, satellite)
- AI-based QKD security (QBER anomaly detection, attack classification)
- AI + 6G physical-layer security
- Hybrid classical-quantum cryptography

05 Mathematics Prerequisite Roadmap

Papers are the wrong tool for learning foundational math — a good course or textbook teaches it far more efficiently. Papers only enter once you can already read the notation. Recommended resources over papers for this section:

Topic	Needed for	Recommended source (not a paper)
Linear algebra	Everything from Week 5 onward (esp. attention math)	3Blue1Brown 'Essence of Linear Algebra'; Gilbert Strang (MIT 18.06)
Calculus & gradients	Backpropagation, optimization	3Blue1Brown 'Essence of Calculus'; Khan Academy multivariable calc
Probability & statistics	Evaluation metrics, loss functions, QBER statistics	Harvard Stat 110 (Blitzstein) lectures/notes
Optimization	Gradient descent, Adam, convexity	Boyd & Vandenberghe, 'Convex Optimization', Ch. 1–3 (as reference, not full read)

Mastery checkpoint: you can derive the gradient of mean-squared-error loss with respect to a linear model's weights on paper, without looking it up.

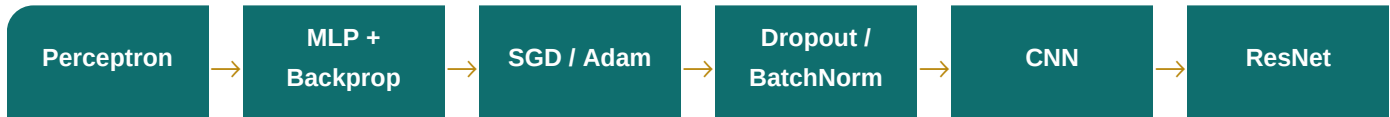
06 Classical ML Research Foundation

Classical ML is learned from textbooks/scikit-learn documentation, not papers — the algorithms are decades old and better taught as engineering skills. The one place a paper adds value here is evaluation methodology, because it resurfaces directly in Week 12's BB84 classifier comparisons.

- **Implement, don't just read:** Linear/Logistic Regression, Decision Trees, Random Forest, SVM, K-means, PCA — each in under 50 lines using scikit-learn, then compare against a from-scratch NumPy version of at least one (logistic regression is the standard choice).
- **Evaluation:** know precisely when accuracy is misleading (class imbalance — directly relevant to BB84 attack-vs-normal datasets, which are usually imbalanced) and when to prefer F1 or ROC-AUC.
- **No paper is Tier 1 here.** If you want one grounding reference, skim (not deep-read) the scikit-learn documentation's 'Model evaluation' guide instead of a paper.

07 Deep Learning Research Foundation

This is where papers start to matter, because the ideas (backprop, regularization, batch norm) are the direct ancestors of everything in Weeks 6–10.



PyTorch mechanics to master this week: `Tensor`, `Dataset/DataLoader`, `nn.Module`, `autograd`, `optimizer.step()`, a full training + validation loop, and moving tensors to GPU. Build a small CNN on CIFAR-10 or MNIST before touching ResNet in Week 6 — you need a working baseline to compare ResNet against.

08 NLP Research Evolution

Studied in Week 4 alongside the AI Resume Analyzer build. Teach every step as 'what broke, what fixed it' rather than isolated facts.



Efficient Estimation of Word Representations in Vector Space (Word2Vec) (2013)

TIER 2 · IMPORTANT ■ Conceptual Read · ■ Implement (small)

Authors: Mikolov, Chen, Corrado, Dean | **Venue:** arXiv / ICLR Workshop | **Link:** arxiv.org/abs/1301.3781 | **Code:** word2vec (Google, archived)

Why now: Establishes that meaning can be represented as a dense vector via a prediction task — the conceptual root of every embedding used later (sentence embeddings, RAG retrieval, quantum-feature embeddings in Week 12).

Prerequisites: Basic neural network idea; none of the math is deep. | **Follow-up:** Seq2Seq (Sutskever 2014).

Sequence to Sequence Learning with Neural Networks (2014)

TIER 2 · IMPORTANT ■ Conceptual Read

Authors: Sutskever, Vinyals, Le | **Venue:** NeurIPS | **Link:** arxiv.org/abs/1409.3215 | **Code:** Not found (original)

Why now: Shows the encoder-decoder pattern and its failure mode (a single fixed-length vector bottleneck) — the exact problem attention was invented to fix.

Prerequisites: Word2Vec-level embedding intuition; RNN basics. | **Follow-up:** Neural Machine Translation by Jointly Learning to Align and Translate (Bahdanau 2014).

Neural Machine Translation by Jointly Learning to Align and Translate (2014)

TIER 2 · IMPORTANT ■ Deep Read (attention mechanism only) · ■ Implement

Authors: Bahdanau, Cho, Bengio | **Venue:** ICLR 2015 | **Link:** arxiv.org/abs/1409.0473 | **Code:** Not found (original)

Why now: First working attention mechanism — read this before 'Attention Is All You Need' so self-attention doesn't feel like it appeared from nowhere.

Prerequisites: Seq2Seq encoder-decoder architecture. | **Follow-up:** Attention Is All You Need (Vaswani et al. 2017) — Week 7 deep study.

09 ResNet Research Journey (Week 6 Deep Study)

Before opening the paper: understand the problem it solves.

- **Before ResNet:** stacking more convolutional layers should only help (or at worst plateau) training accuracy — more capacity, more expressive power.
- **The degradation problem:** in practice, past a certain depth, plain (non-residual) networks got *worse* training accuracy, not just worse test accuracy — so it wasn't overfitting, it was an optimization failure.
- **The residual idea:** instead of asking a stack of layers to learn a full mapping $H(x)$, ask it to learn the residual $F(x) = H(x) - x$, and add the input back: $\text{output} = F(x) + x$.
- **Why this helps:** an identity mapping is now trivial to represent (just drive $F(x)$ toward zero), so adding depth can never make optimization strictly harder than the shallower network.

Deep Residual Learning for Image Recognition (ResNet)

TIER 1 · MUST READ ■ Deep Read · ■ Implement · ■ Reproduce

Authors: He, Zhang, Ren, Sun (Microsoft Research) | **Year:** 2015 | **Venue:** CVPR 2016

Link: arxiv.org/abs/1512.03385 | **Official code:** github.com/KaimingHe/deep-residual-networks

Central Idea

Intuitively: it is easier to nudge a near-identity function than to learn a complex mapping from scratch. Mathematically: a residual block computes $y = F(x, \{W_i\}) + x$, where the shortcut connection performs identity mapping and adds no extra parameters or compute.

Mathematical Concepts

- Residual mapping $F(x) = H(x) - x$ vs. direct mapping $H(x)$
- Identity shortcut connections (element-wise addition, requires matching dimensions or a 1×1 projection when they don't match)
- Bottleneck blocks ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ convolutions) for deeper variants

Architecture

Stacks of residual blocks organized into stages of increasing channel depth and decreasing spatial resolution (18/34-layer 'basic block' and 50/101/152-layer 'bottleneck block' variants), ending in global average pooling and a linear classifier.

Prerequisites

CNN basics, backpropagation, batch normalization.

Dependency

Plain deep CNN baselines (VGG-style) that this paper explicitly compares against.

Follow-up

Wide ResNet, DenseNet, and eventually the shift toward attention-based vision backbones (ViT) — useful context, not required reading now.

Sections to Read Deeply

Section 3 (Deep Residual Learning) and Section 4.1 (experiments comparing plain vs. residual networks on ImageNet/CIFAR-10).

Sections to Skim

Related work (Section 2); detection/segmentation transfer results at the end.

Implementation

A baseline plain CNN and a ResNet of matched depth in PyTorch on CIFAR-10.

Experiment

Reproduce the core claim: plain-34 trains worse than plain-18, but ResNet-34 trains better than ResNet-18.

Ablation

Remove the residual (shortcut) connections from your ResNet implementation and retrain with identical hyperparameters. Compare training loss curves, validation accuracy, and convergence speed against the residual version.

Research Questions

- Does the benefit of residual connections shrink as network depth decreases?
- Do residual connections change the loss landscape's smoothness, or just its optimizability?

Limitations

Doesn't fully explain *why* optimization improves (later work — e.g. loss-landscape visualization papers — addresses this); very deep ResNets still show diminishing returns without further tricks (stochastic depth, better normalization).

Researcher Lesson

The paper's power comes from one extremely clean, isolatable experiment (plain vs. residual, everything else held constant). When you design your own BB84 ablations in Week 12, copy this discipline: change exactly one thing at a time.

10 Transformer Research Journey (Week 7 Deep Study)

This should be the single deepest paper study in the roadmap — everything in Weeks 8–10 depends on genuinely understanding it, not just recognizing the diagram.

Attention Is All You Need

TIER 1 · MUST READ ■ Deep Read · ■ Implement

Authors: Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin (Google) | **Year:** 2017 | **Venue:** NeurIPS 2017

Link: arxiv.org/abs/1706.03762 | **Official code:** github.com/tensorflow/tensor2tensor

Historical Context

Recurrent (Seq2Seq + attention) models processed tokens sequentially, which is slow to train (no parallelism across time) and still struggled with very long-range dependencies. This paper asks: can attention alone, with no recurrence, do the job?

Central Idea

Intuitively: let every token directly 'look at' every other token and decide how much to weight each one, instead of passing information step-by-step through a hidden state. Technically: replace recurrence entirely with self-attention layers plus position-wise feed-forward networks.

Mathematical Concepts

- Scaled dot-product attention: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$
- Why the $1/\sqrt{d_k}$ scaling matters: without it, dot products grow large in magnitude as d_k increases, pushing softmax into regions with vanishingly small gradients
- Multi-head attention: project Q, K, V into h subspaces, attend in parallel, concatenate, and apply a final output projection — lets the model attend to different representation subspaces simultaneously
- Positional encoding: sinusoidal functions of position, added to embeddings, since self-attention has no inherent notion of order
- Residual connections + layer normalization around every sub-layer

Architecture

Encoder stack (self-attention + feed-forward, repeated N times) and decoder stack (masked self-attention + encoder-decoder cross-attention + feed-forward, repeated N times); masking in the decoder prevents attending to future tokens.

Prerequisites

Vectors/matrices, softmax, embeddings, the Seq2Seq + Bahdanau-attention story from Week 4.

Dependency

Bahdanau attention (Week 4) — read that first so QK■ doesn't appear unmotivated.

Follow-up

BERT (encoder-only pretraining) and GPT (decoder-only pretraining) — Week 8.

Sections to Read Deeply

Section 3 (Model Architecture) in full, including all sub-equations; Section 3.2.1–3.2.3 on attention specifically.

Sections to Skim

Section 4 (Why Self-Attention, complexity comparison table) — useful context, not required to reproduce.

Sections to Skip Initially

Section 6 (Results) hyperparameter tables — revisit only when actually training.

Implementation

A simplified Transformer encoder (and ideally decoder) from scratch in PyTorch — write the attention function yourself before ever importing a pretrained model.

Experiment

Train your from-scratch Transformer on a small sequence task (e.g. copy task or small machine-translation subset) and confirm it converges.

Ablation

Remove positional encoding and observe whether/how performance degrades on an order-sensitive task; try single-head vs. multi-head attention.

Research Questions

- How much of the Transformer's advantage comes from parallelism (training speed) vs. modeling quality (fewer errors) on long sequences?
- Does multi-head attention actually learn distinct, interpretable roles per head on your small task?

Limitations

Quadratic compute/memory in sequence length (a direct motivation for later efficient-attention research); needs large data/compute to realize its benefits, which smaller papers on your track may not have.

Later Research

Efficient/sparse attention variants address the quadratic cost; BERT and GPT show what happens when you pretrain this architecture at scale — Week 8.

Researcher Lesson

The paper's title is also its thesis — it makes one crisp, falsifiable claim ('attention alone is sufficient') and builds the entire architecture and experiment section to test exactly that claim. A strong paper (and a strong ablation you design later) usually reduces to one sentence you could argue for or against.

11 BERT → GPT → LLM Evolution (Week 8)



Transition	Limitation before	New idea	Limitation remaining
Transformer → BERT	No general pretrained language representation for downstream tasks	Bidirectional masked-language-model pretraining + fine-tuning	Not generative — can't produce free-form text
Transformer → GPT-1	Supervised NLP tasks each needed task-specific architectures	Decoder-only, left-to-right autoregressive pretraining + light fine-tuning	Small scale limited generalization
GPT-1/2 → GPT-3	Models still needed task-specific fine-tuning data	Massive scale enables few-shot/zero-shot prompting without weight updates	Outputs not aligned with what users actually want; hallucination
GPT-3 → InstructGPT	Raw pretraining objective ≠ 'be helpful and honest'	Supervised fine-tuning + RLHF (reward model + PPO) on human preferences	RLHF is complex/unstable to train — motivates DPO (Week 10)

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

TIER 1 · MUST READ ■ Deep Read · ■ Conceptual (fine-tuning heads) · ■ Implement (use pretrained)

Authors: Devlin, Chang, Lee, Toutanova (Google AI Language) | **Year:** 2018 | **Venue:** NAACL 2019

Link: arxiv.org/abs/1810.04805 | **Official code:** github.com/google-research/bert

Why Read It

The canonical example of 'pretrain then fine-tune' for NLP, and the direct ancestor of the sentence-embedding models your RAG system (Week 9) will use for retrieval.

Core Contribution

Masked Language Modeling (MLM: predict randomly masked tokens using both left and right context) plus Next Sentence Prediction, pretrained on large unlabeled text, then fine-tuned with a small added head per downstream task.

Mathematical Concepts

Cross-entropy loss over masked token predictions; the encoder-only Transformer stack from Week 7, unchanged.

Prerequisites

Transformer encoder architecture (Week 7).

Implementation

Fine-tune a pretrained BERT (via Hugging Face) on a small classification task; do not attempt to pretrain BERT from scratch — that is a compute exercise, not a learning one.

Limitations

Bidirectionality makes it unsuitable for free-form generation; fixed input length; two-stage pretrain/fine-tune workflow is less flexible than GPT-style prompting.

Later Research

RoBERTa (better-tuned pretraining), and — more importantly for this roadmap — the shift toward decoder-only, prompted LLMs (GPT family) that avoided per-task fine-tuning altogether.

Researcher Lesson

BERT's ablation section (which pretraining objective matters, does NSP actually help) is a good second reference model for how to design your own BB84 ablations in Week 12.

Language Models are Few-Shot Learners (GPT-3) (2020)

TIER 1 · MUST READ ■ Deep Read (Sections 1–3) · ■ Skim (task-specific results)

Authors: Brown et al. (OpenAI) | **Venue:** NeurIPS 2020 | **Link:** arxiv.org/abs/2005.14165 | **Code:** Not found (weights not released)

Why now: Establishes in-context/few-shot learning as an emergent property of scale — the conceptual basis for prompting, which your Resume Analyzer and Research Assistant projects both depend on.

Prerequisites: GPT-1/GPT-2 decoder-only pretraining idea. | **Follow-up:** Training language models to follow instructions (InstructGPT, Ouyang et al. 2022).

Training Language Models to Follow Instructions with Human Feedback (InstructGPT)

TIER 1 · MUST READ ■ Deep Read · ■ Conceptual (PPO internals)

Authors: Ouyang, Wu, Jiang, Almeida, Wainwright, et al. (OpenAI) | **Year:** 2022 | **Venue:** NeurIPS 2022
Link: arxiv.org/abs/2203.02155 | **Official code:** Not found

Why Read It

This is the paper behind 'ChatGPT-style' behavior — it explains exactly why raw pretrained LLMs are unhelpful/unaligned by default and what three-stage recipe fixes it.

Core Contribution

A three-stage pipeline: (1) supervised fine-tuning on human-written demonstrations, (2) train a reward model on human rankings of model outputs, (3) fine-tune the policy against the reward model using PPO (reinforcement learning).

Mathematical Concepts

Bradley-Terry preference model for the reward model; the PPO clipped-objective (conceptual understanding is sufficient — deriving PPO is out of scope for this roadmap).

Architecture / Algorithm

GPT-3-scale base model → SFT → reward model → RL fine-tuning with a KL penalty against the SFT policy to prevent reward over-optimization.

Prerequisites

GPT-3's pretraining/few-shot story.

Implementation

Full RLHF is out of reach on a student budget — instead, implement DPO (Week 10), which achieves a similar effect without a separate RL loop.

Limitations

Expensive (needs a reward model plus RL infra); reward hacking; smaller 'aligned' models can still be less capable on raw benchmarks than larger unaligned ones.

Later Research

Direct Preference Optimization (DPO, 2023) reformulates the same objective as a single supervised loss, removing the RL loop — read immediately after this.

Researcher Lesson

The paper is explicit that 'bigger is not automatically better aligned' — a reminder that your BB84 classifier comparisons (Week 12) should also report failure modes, not just an accuracy number.

12 Information Retrieval → RAG Evolution (Week 9)



TF-IDF and BM25 are studied from documentation/tutorials (e.g. scikit-learn, the rank_bm25 library), not papers — they are classical statistics, not research findings you need to critique.

Dense Passage Retrieval for Open-Domain Question Answering (DPR) (2020)

TIER 2 · IMPORTANT ■ Deep Read · ■ Implement

Authors: Karpukhin, Ouz, Min, Lewis, Wu, Edunov, Chen, Yih (Meta AI) | **Venue:** EMNLP 2020 | **Link:** arxiv.org/abs/2004.04906 | **Code:** github.com/facebookresearch/DPR

Why now: Shows that a dual-encoder trained with in-batch negatives beats BM25 for open-domain retrieval — this is exactly the retriever architecture your AI Research Assistant needs.

Prerequisites: BERT-style encoders; cosine/dot-product similarity. | **Follow-up:** REALM (Guu et al. 2020), then RAG (Lewis et al. 2020).

REALM: Retrieval-Augmented Language Model Pre-Training (2020)

TIER 2 · IMPORTANT ■ Conceptual Read

Authors: Guu, Lee, Tung, Pasupat, Chang (Google Research) | **Venue:** ICML 2020 | **Link:** arxiv.org/abs/2002.08909 | **Code:** Not found (original)

Why now: First to integrate retrieval directly into pretraining rather than bolting it on at inference — conceptually important, but you will implement the inference-time pattern (RAG), not this one.

Prerequisites: DPR-level dense retrieval. | **Follow-up:** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (RAG, 2020).

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (RAG)

TIER 1 · MUST READ ■ Deep Read · ■ Implement · ■ Reproduce (small scale)

Authors: Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, et al. (Meta AI) | **Year:** 2020 | **Venue:** NeurIPS 2020
Link: arxiv.org/abs/2005.11401 | **Official code:** github.com/huggingface/transformers (RAG implementation)

Why Read It

This is the architecture your AI Research Assistant project directly implements — read it as a build spec, not just theory.

Core Contribution

Combine a pretrained dense retriever (DPR-style) with a pretrained seq2seq generator, marginalizing over top-k retrieved documents, and fine-tune end-to-end.

Mathematical Concepts

RAG-Sequence vs. RAG-Token formulations — probability of an output sequence marginalized over retrieved documents, $p(y|x) = \sum_z p(z|x) p(y|x,z)$.

Architecture

Query → dense retriever → top-k passages from a vector index → generator conditions on (query + retrieved passages) → output.

Prerequisites

DPR-level retrieval; a Transformer-based generator (BERT/GPT-level understanding).

Implementation

Build the full pipeline: PDF → chunking → embeddings → vector database → retriever → (optional reranker) → LLM → answer, exactly as specified in the Week 9 project.

Experiment

Compare answer quality/groundedness with retrieval on vs. off on the same question set — a direct, cheap reproduction of the paper's core claim.

Ablation

Vary top-k (number of retrieved chunks) and chunk size; measure retrieval precision and answer quality.

Limitations

Retrieval quality caps generation quality (garbage in, garbage out); no guarantee the generator actually uses the retrieved evidence faithfully.

Later Research

Reranking (cross-encoders) to fix retrieval precision; modern agentic RAG with multi-hop retrieval and tool use.

Researcher Lesson

RAG's central bet is that retrieval and generation are separable problems you can solve independently and then compose — a generally useful research strategy: can a hard problem be decomposed into two problems each of which is already well understood?

13 Modern LLM Research Landscape (Week 10)

Not every LLM paper is worth deep-reading. This table ranks by what your goals actually require.

Paper	Classification	Why
LoRA (Hu et al. 2021)	MUST DEEPLY READ	You will fine-tune models on limited GPU budget — this is the technique that makes that possible.
QLoRA (Dettmers et al. 2023)	MUST DEEPLY READ	Combines LoRA with 4-bit quantization — the realistic way to fine-tune on a single consumer GPU or free-tier cloud GPU.
DPO (Rafailov et al. 2023)	READ IF TIME	Simpler alternative to RLHF; useful conceptually, implement only if a project needs preference-tuning.
Scaling Laws (Kaplan et al. 2020)	CONCEPTUAL UNDERSTANDING	Explains why 'bigger models + more data' works — you need the intuition, not the exact fitted curves.
Chinchilla (Hoffmann et al. 2022)	CONCEPTUAL UNDERSTANDING	Corrects Kaplan's scaling law (compute-optimal models need more data, not just more parameters) — read as a two-paragraph update to the above.
GPT-4 / o1-style technical reports	SKIM ONLY	Sparse on technical detail by design (closed models); useful for capability awareness, not implementable learning.
Full RLHF/PPO derivations	DO NOT READ YET	Requires RL background you have not built; DPO gives you 90% of the practical value for 10% of the prerequisite cost.

LoRA: Low-Rank Adaptation of Large Language Models (2021)

TIER 2 · IMPORTANT ■ Deep Read · ■ Implement

Authors: Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, Chen (Microsoft) | **Venue:** ICLR 2022 | **Link:** arxiv.org/abs/2106.09685 | **Code:** github.com/microsoft/LoRA

Why now: Freezes pretrained weights and injects small trainable low-rank matrices into attention layers — cuts fine-tuning cost by orders of magnitude, which is the only realistic way you will fine-tune an LLM as a student.

Prerequisites: Transformer attention (Week 7); basic linear algebra rank intuition. | **Follow-up:** QLoRA (Dettmers et al. 2023).

QLoRA: Efficient Finetuning of Quantized LLMs (2023)

TIER 2 · IMPORTANT ■ Conceptual Read · ■ Implement

Authors: Dettmers, Pagnoni, Holtzman, Zettlemoyer | **Venue:** NeurIPS 2023 | **Link:** arxiv.org/abs/2305.14314 | **Code:** github.com/artidoro/qlora

Why now: 4-bit quantization (NF4) + LoRA lets you fine-tune multi-billion-parameter models on a single GPU — directly enables Project 4's LLM component on limited hardware.

Prerequisites: LoRA. | **Follow-up:** Direct Preference Optimization (2023) if you want to go further into alignment.

Direct Preference Optimization (DPO) (2023)

TIER 2 · IMPORTANT ■ Conceptual Read

Authors: Rafailov, Sharma, Mitchell, Ermon, Manning, Finn (Stanford) | **Venue:** NeurIPS 2023 | **Link:** arxiv.org/abs/2305.18290 | **Code:** github.com/eric-mitchell/direct-preference-optimization

Why now: Reformulates RLHF's objective into a single classification-style loss on preference pairs — no reward model, no RL loop. Read to understand alignment without needing PPO.

Prerequisites: InstructGPT/RLHF conceptual understanding. | **Follow-up:** None required for this roadmap — this is a stopping point, not a launching point.

14 Research Methodology

These are the concepts Week 11 formalizes, but they are used implicitly from Week 6 onward (every ablation you run is a small research-methodology exercise).

- **Research question:** a specific, answerable question — not a topic. 'Can temporal QBER features detect stealthy eavesdropping below the fixed threshold?' is a research question; 'QKD security' is a topic.
- **Hypothesis:** a falsifiable prediction ('a Random Forest on temporal QBER features will detect X% more stealthy attacks than a fixed-threshold detector').
- **Baseline:** the simplest reasonable method your new method must beat — for BB84, this is usually the fixed QBER-threshold detector.
- **Dataset:** where it comes from, how realistic it is, and whether it leaks information between train/test splits.
- **Experimental design:** what varies, what is held constant, and why.
- **Metrics:** chosen for the problem, not for flattering the result (e.g. FNR/FPR matter more than raw accuracy for imbalanced attack-detection data).
- **Ablation:** remove or change exactly one component and re-measure — isolates **why** something works, not just **whether** it works.
- **Reproducibility:** could someone else, given your paper/code, get the same result?
- **Statistical significance:** is the improvement bigger than run-to-run noise (multiple seeds)?
- **Error analysis:** look at the specific cases the model gets wrong, not just the aggregate score.
- **Limitations & negative results:** stating what didn't work is part of the research, not a failure to hide.
- **Research gap:** a limitation in the existing literature that is not yet addressed — the starting point for your own contribution.

15 Paper Reading Methodology

Never read a paper linearly from page 1. Use five passes, each with a different goal:

Pass 1 — Orientation

Title, abstract, introduction, figures, conclusion. Goal: can you state the paper's one-sentence claim?

Pass 2 — Technical Understanding

Method, architecture, equations, algorithm, experimental setup. Goal: could you redraw the architecture from memory?

Pass 3 — Critical Analysis

Baselines, assumptions, ablations, results, limitations, reproducibility. Goal: what would you challenge if you were the reviewer?

Pass 4 — Implementation

Reconstruct the algorithm, implement a simplified version, reproduce an experiment. Goal: does your implementation actually behave like the paper claims?

Pass 5 — Research Thinking

What remains unsolved? What assumption could be relaxed? What baseline is missing? Can this be applied to QKD? Can ML improve it, or can the quantum component be removed entirely? Goal: generate at least one research question of your own.

Mastery Criteria — do not move on until you can:

- Explain the paper without notes.
- Explain the problem it solves and why previous approaches were insufficient.
- Explain the central contribution and derive/understand its core equation.
- Reconstruct the architecture and implement a simplified version.
- Explain why it works, identify its limitations, and explain later improvements.
- Propose at least one meaningful research question that follows from it.

16 Complete 12-Week Roadmap

Full paper-level detail for Weeks 6-10 and 12 is in Sections 9-13 and 19 — this table is the week-by-week synthesis so you can see the whole 90 days at a glance.

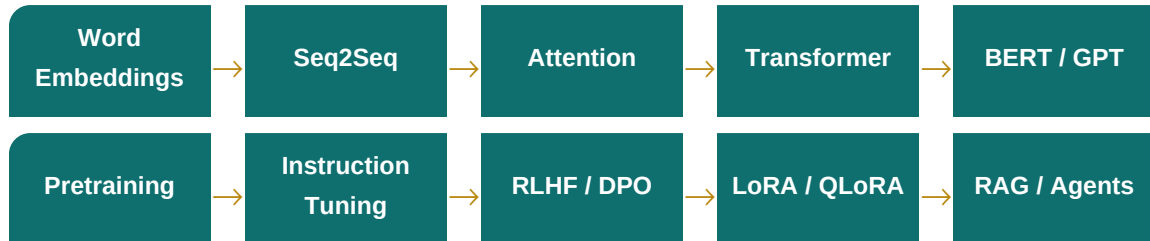
Wk	Focus	Key Papers	Milestone	Checkpoint	Deliverable
1	Python + Computational Foundation Python, Git/GitHub, NumPy, Pandas, Matplotlib, APIs, JSON, exceptions, venvs, type hints	None (textbooks/docs, not papers)	Set up repo structure for all 5 future projects	Write a typed, tested Python script that calls a public API and handles errors gracefully	GitHub repo with README + CI-ready structure
2	Mathematics + Classical ML Linear algebra, calculus, probability, statistics, optimization; regression, trees, SVM, clustering, PCA, evaluation metrics	None (see Section 5-6 for resources)	Notebook comparing 4 classical models on one dataset	Derive gradient descent for linear regression on paper from scratch	Jupyter notebook + written derivation
3	FastAPI + Databases + AI Engineering REST, Pydantic, validation, auth, async; PostgreSQL schema design, joins, indexes, CRUD	None (implementation-first week)	Skeleton FastAPI + PostgreSQL backend for Project 1	Backend passes CRUD + auth integration tests	Working API with OpenAPI docs
4	AI Resume Analyzer + NLP Foundation Text extraction, embeddings, semantic similarity, sentence embeddings, IR basics	Tier 2: Word2Vec, Seq2Seq, Bahdanau Attention (Sec. 8)	PROJECT 1: AI Resume Analyzer (parsing → LLM → matching → scoring)	Explain why embeddings beat keyword matching for JD-resume matching	Deployed Resume Analyzer + notebook entries for 3 papers
5	Deep Learning + PyTorch Neural networks, backprop, SGD/Adam, regularization, dropout, batchnorm, CNNs; full PyTorch training loop	None (foundation week; ResNet paper starts Week 6)	Baseline CNN on CIFAR-10/MNIST in PyTorch	Explain backprop through a 2-layer network without notes	Trained baseline CNN + training-curve plots
6	ResNet Deep Study Degradation problem, residual connections, identity mapping	Tier 1: Deep Residual Learning for Image Recognition (Sec. 9)	PROJECT 2: ResNet vs. plain CNN + no-residual ablation	Reproduce: plain-34 underperforms plain-18; ResNet-34 beats ResNet-18	Ablation report: accuracy, convergence, training time
7	Transformers Q/K/V, scaled dot-product attention, multi-head attention, positional encoding, encoder/decoder, residuals, layer norm	Tier 1: Attention Is All You Need (Sec. 10)	PROJECT 3: Transformer from scratch in PyTorch	Derive scaled dot-product attention equation and explain the $1/\sqrt{d_k}$ scaling	From-scratch Transformer trained on a toy sequence task
8	NLP → Transformer → BERT → GPT → LLM Pretraining, bidirectional vs. autoregressive modeling, few-shot learning, RLHF	Tier 1: BERT, GPT-3, InstructGPT (Sec. 11)	Fine-tune a small pretrained BERT/GPT model on a downstream task	Narrate the full Word2Vec→InstructGPT evolution as one continuous story	Fine-tuned model + 3 completed research-notebook entries

W k	Focus	Key Papers	Milestone	Checkpoint	Deliverable
9	RAG + AI Research Assistant TF-IDF/BM25 → dense retrieval → DPR → REALM → RAG → reranking; vector databases	Tier 1: RAG; Tier 2: DPR, REALM (Sec. 12)	PROJECT 4: AI Research Assistant (paper upload → RAG → Q&A;)	Show retrieval-on vs. retrieval-off answer quality difference on your own test questions	Deployed Research Assistant (Next.js + FastAPI + vector DB + Docker)
10	Modern LLM Research + LLM Engineering Scaling laws, LoRA/QLoRA, DPO, quantization, hallucination, agents/tool use	Tier 2: LoRA, QLoRA, DPO, Scaling Laws (Sec. 13)	Fine-tune a small open model with LoRA/QLoRA on a task from Project 1 or 4	Explain why LoRA reduces trainable parameters without hurting quality much	LoRA fine-tuning notebook + before/after comparison
11	Research Methodology + Paper Reproduction Research question, hypothesis, baselines, ablation, reproducibility, error analysis	Reproduction target selected per Section 18 criteria	Full reproduction plan executed end-to-end (see Section 18)	Present discrepancy analysis between your results and the paper's reported results	Reproduction report (paper-style write-up)
12	AI + QKD + 6G Specialization QBER, channel noise, attack simulation, feature engineering, classical ML vs. deep learning for security classification	Tier 3: QKD-as-QML, QBER-ML detection, QKD anomaly detection, AI+6G PLS survey (Sec. 19)	PROJECT 5: ML-Based Security Analysis of BB84/QKD	Critically evaluate one AI+QKD paper using the 18-question framework in Section 19	BB84 security-classification study + critical-evaluation write-up

17 Paper Dependency Graph

Four evolution chains cover the entire roadmap. Chains are read left to right; each node assumes everything to its left is understood.

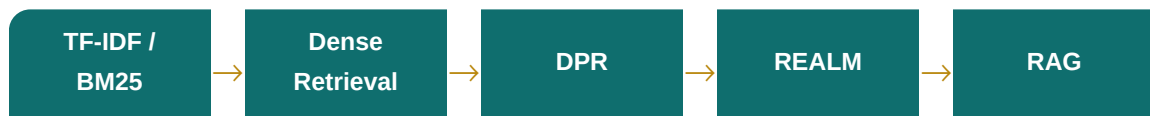
NLP / LLM chain



Vision chain



Information Retrieval → RAG chain



AI + QKD / 6G chain



18 Deep-Read, Implementation & Reproduction Paper Lists

Deep-Read List (Tier 1 — full mastery required)

- Attention Is All You Need — Vaswani et al., 2017 (Week 7)
- Deep Residual Learning for Image Recognition — He et al., 2015 (Week 6)
- BERT — Devlin et al., 2018 (Week 8)
- Language Models are Few-Shot Learners (GPT-3) — Brown et al., 2020 (Week 8)
- Training LMs to Follow Instructions (InstructGPT) — Ouyang et al., 2022 (Week 8)
- Retrieval-Augmented Generation (RAG) — Lewis et al., 2020 (Week 9)

Implementation List (papers you should code, not just read)

- ResNet: baseline CNN + ResNet + no-residual ablation (Week 6)
- Transformer: attention, multi-head attention, positional encoding from scratch (Week 7)
- BERT/GPT: fine-tune a pretrained checkpoint on a downstream task (Week 8)
- RAG: full chunk → embed → retrieve → rerank → generate pipeline (Week 9)
- LoRA/QLoRA: fine-tune a small open model on limited hardware (Week 10)

Reproduction List (candidates for the Week 11 full reproduction)

Selection criteria: your current knowledge, available compute, implementation difficulty, scientific value, and relevance to your AI+QKD specialization — not fame.

Candidate	Compute needed	Relevance to your track	Recommended?
ResNet degradation ablation	Low (CIFAR-10, single GPU/Colab)	Direct rehearsal for the ablation discipline you need in Week 12	YES — best fit
Transformer toy-task training	Low-medium	Core to Weeks 7-10; already partly built	Good alternative
Full BERT/GPT-3 pretraining	Very high (out of reach)	Not needed — fine-tuning a checkpoint gives the same learning	NO
A Tier-3 AI+QKD ML paper (e.g. QBER-based attack detection)	Low (simulated BB84 data)	Highest direct relevance to Project 5 and your final-year project	YES — do this as Project 5, informally reproducing its pipeline

Recommendation: use the ResNet ablation as your formal Week-11 reproduction exercise (clean, well-scoped, cheap to run, and it directly trains the ablation methodology), and treat Project 5 (Week 12) as an applied reproduction of the temporal-QBER machine-learning framework's *pipeline* (features → classical ML → attack detection) on your own simulated BB84 data.

19 AI + Quantum / QKD / 6G Research Roadmap

This section connects everything above to your existing final-year project: *Secure 6G Communication Using QKD / BB84*.

Project 5 Pipeline: ML-Based Security Analysis of BB84/QKD



- **Simulate** BB84 with configurable channel loss, noise, and at least 2-3 distinct attack types (e.g. intercept-resend, photon-number-splitting-style, calibration attack).
- **Extract features** beyond raw session-level QBER: temporal QBER variation, basis-dependent asymmetry, burst behavior, channel-loss interaction — session-level QBER alone misses stealthy attacks that stay under the fixed ~11% threshold.
- **Compare models:** Logistic Regression, Random Forest, XGBoost, SVM, and a small neural network/LSTM only where the literature justifies it (e.g. attacks with genuine temporal structure).
- **Evaluate** with accuracy, precision, recall, F1, ROC-AUC, and — critically — False Negative Rate, since missing an attack is far more costly than a false alarm.
- **Ablate:** remove the QBER feature, remove channel-loss features, remove temporal features; compare classical ML vs. deep learning; test robustness under changing channel parameters.

Tier 3 & 4 Papers — AI + QKD / 6G Specialization

These are recent (2024-2026) papers, verified via direct search rather than recalled from memory, since this is a fast-moving specialization area.

Quantum Cryptography (original BB84 protocol) (1984 / republished 2014)

TIER 3 · SPECIALIZATION ■ Deep Read (you likely already know this from your final-year project)

Authors: Bennett, Brassard | **Venue:** IEEE Int'l Conf. on Computers, Systems & Signal Processing (1984); reprinted in Theoretical Computer Science, 2014 | **Link:** doi.org/10.1016/j.tcs.2014.05.025 (2014 reprint); no open arXiv version of the original | **Code:** Not found (predates modern code-sharing norms)

Why now: The protocol every paper in this section assumes you already understand deeply — your existing project advantage.

Prerequisites: Quantum bit (qubit) basics, polarization/basis encoding, classical error-correction and privacy amplification at a conceptual level. | **Follow-up:** Any AI+QKD security paper below.

QKD as a Quantum Machine Learning Task (2024)

TIER 3 · SPECIALIZATION ■ Conceptual Read

Authors: Decker, Gallezot, Kerstan, Paesano, Ginter, Wormsbecher | **Venue:** arXiv (quant-ph), rev. Feb 2025 | **Link:** arxiv.org/abs/2410.01904 | **Code:** Not found

Why now: Frames optimizing an eavesdropping attack on BB84 as a QML optimization problem — good test case for the 'is the quantum component genuinely necessary?' critical-evaluation question below.

Prerequisites: BB84; basic parameterized-quantum-circuit intuition (can be conceptual, not derived). | **Follow-up:** Deep-learning-based continuous attacks on QKD protocols (2024).

Deep-Learning-Based Continuous Attacks on QKD Protocols (2024)

TIER 3 · SPECIALIZATION ■ Conceptual Read

Authors: (2024 preprint) | **Venue:** arXiv (quant-ph) | **Link:** arxiv.org/abs/2408.12571 | **Code:** Not found
Why now: Uses deep learning for quantum state tomography to build a continuous-variable attack — a genuine case where the ML component targets a physical measurement problem, worth contrasting with attack-*detection* papers below.
Prerequisites: BB84; basic idea of quantum state tomography (conceptual). | **Follow-up:** Deep Anomaly Detection for Active Attacks on the Receiver in QKD (2025).

Deep Anomaly Detection for Active Attacks on the Receiver in QKD (2025)

TIER 3 · SPECIALIZATION ■ Deep Read · ■ Implement (methodology)

Authors: (2025 preprint) | **Venue:** arXiv (cs.CR / quant-ph) | **Link:** arxiv.org/abs/2508.12749 | **Code:** Not found
Why now: Deep SVDD-based anomaly detection trained only on secure-operation data (one-class learning) — directly reusable methodology for Project 5, since labeled attack data is usually scarce/simulated.
Prerequisites: Classical ML anomaly detection concepts; autoencoders/one-class SVM intuition. | **Follow-up:** Beyond the QBER Threshold (2026), for a supervised multi-attack alternative.

Beyond the QBER Threshold: A Temporal QBER-Based ML Framework for Multi-Attack Detection in BB84 QKD (2026)

TIER 3 · SPECIALIZATION ■ Deep Read · ■ Implement · ■ Reproduce (pipeline)

Authors: Isha, Singh, Kumar, Pal, Hambarde, Shukla | **Venue:** arXiv (cs.CR) | **Link:** arxiv.org/abs/2608.04047 | **Code:** Not found
Why now: The closest existing paper to Project 5's design: 63 physics-informed temporal/channel features, Random Forest/XGBoost/SVM-RBF classifiers, SHAP explainability, evaluated against a fixed-threshold baseline on seven attack types. Use its feature-engineering approach as your Project 5 starting point.
Prerequisites: QBER concept; classical ML classifiers; SHAP-style feature attribution (conceptual). | **Follow-up:** Machine Learning for Specialized QKD Aspects (survey, 2026) for the wider landscape.

Adversarial Reinforcement Learning for Adaptive Eavesdropping in BB84 QKD (2026)

TIER 4 · FRONTIER ■ Skim

Authors: (2026 preprint) | **Venue:** arXiv (quant-ph) | **Link:** arxiv.org/abs/2606.22962 | **Code:** Not found
Why now: Frontier/adversarial framing — an RL-based adaptive attacker. Useful to know the field is moving toward adaptive attacks, but requires RL background beyond this roadmap's scope; revisit after Week 10's LLM-RL exposure if this direction becomes your thesis focus.
Prerequisites: Reinforcement learning basics (not covered in this 90-day plan). | **Follow-up:** Defer until RL fundamentals are covered separately.

Machine Learning for Specialized QKD Aspects: A Survey of Adaptive Protocols, Free-Space Links, 6G Integration, and Steerability-Aware Security (2026)

TIER 4 · FRONTIER ■ Skim (use as a map, not a deep read)

Authors: Al-Mohammed, Al-Ali | **Venue:** arXiv (quant-ph) | **Link:** arxiv.org/abs/2608.08280 | **Code:** Not found
Why now: A survey — read it to find your specific research gap (Section 20), not to deep-read every cited sub-paper. Directly covers free-space/6G-integrated QKD, which is your final-year project's exact context.
Prerequisites: Everything above in this section. | **Follow-up:** Pick 1-2 papers it cites that match your Project 5 direction and deep-read those instead.

Deep Learning-Aided 6G Wireless Networks: A Comprehensive Survey of Revolutionary PHY Architectures (2022)

TIER 3 · SPECIALIZATION ■ Skim (physical-layer sections only)

Authors: Ozpoyraz, Dogukan, Gevez, Altun, Basar | **Venue:** arXiv / IEEE Access | **Link:** arxiv.org/abs/2201.03866 | **Code:** Not found (survey)

Why now: Grounds the '6G' half of your project title in real physical-layer AI research (channel estimation, signal detection) rather than buzzword-level 'AI + 6G' claims.

Prerequisites: Basic wireless communication (you already have this from E&TC; coursework). | **Follow-up:** Security and Privacy for 6G: A Survey (2021), for the security-specific angle.

Security and Privacy for 6G: A Survey on Prospective Technologies and Challenges (2021)

TIER 3 · SPECIALIZATION ■ Skim (Sections on physical-layer security & quantum-safe comms)

Authors: Nguyen et al. | **Venue:** IEEE Communications Surveys & Tutorials | **Link:** arxiv.org/abs/2108.11861 | **Code:** Not found (survey)

Why now: Explicitly places quantum-safe communication and QKD within the broader 6G security landscape — useful for framing your final-year project's motivation section correctly.

Prerequisites: None beyond general wireless/security awareness. | **Follow-up:** None required — use for framing, not implementation.

Critical Evaluation Framework for AI + QKD Papers

Apply every one of these questions to every AI+QKD/quantum-security paper you read. The goal is to distinguish genuine AI+quantum research from 'AI applied to quantum' as a label.

- Is the quantum component genuinely necessary, or could the same problem be solved classically?
- Is there a claimed quantum advantage, and is it theoretically justified (not just asserted)?
- Is the quantum system simulated or built on real hardware/optical links?
- Is the dataset realistic, and are the attack models realistic (not toy assumptions)?
- Is the ML problem well-defined, with a clear input/output and label source?
- Are the baselines strong (a fixed-threshold detector is weak; a tuned classical model is stronger)?
- Is there data leakage between train/test splits (common with synthetic time-series data)?
- Is the evaluation statistically meaningful (multiple seeds/runs, not one lucky split)?
- Is the security claim actually demonstrated, or just plausible-sounding?
- Is QBER being interpreted correctly (session-level vs. temporal, threshold vs. distribution)?
- Is the paper reproducible, and is code actually available?
- What assumptions are hidden (channel model, attacker capability, noise distribution)?
- What could become a publishable extension of this specific paper?

20 Research Gap Identification Framework



Research-gap categories to check against the AI+QKD/6G literature (only claim a gap that is actually supported by what you read — do not invent one to justify a project):

- Dataset limitations — most public AI+QKD datasets are simulated; is that acknowledged and bounded?
- Simulation-to-reality gap — does the noise/channel model reflect real fiber/free-space physics?
- Weak baselines — is the fixed-QBER-threshold detector the only comparison offered?
- Unrealistic attack assumptions — does the attacker model cover only textbook attacks?
- Limited channel models — single fixed loss/noise value instead of a swept range?
- Poor generalization — trained/tested on one channel configuration only?
- Lack of robustness testing — no evaluation under drifting channel conditions?
- Insufficient ablation — no feature-removal or model-comparison study?
- Lack of interpretability — black-box classifier with no SHAP/feature-importance analysis?
- Computational inefficiency — is the method deployable at QKD's real-time timescales?
- Lack of real-world validation — simulation only, no hardware/testbed result?
- Classical-vs-quantum comparison missing — is the 'AI+quantum' framing ever benchmarked against a purely classical detector?
- Security claims not sufficiently demonstrated — detection accuracy reported, but not tied back to a formal security proof or key-rate impact?

21 Project Roadmap

Project 1 — AI Resume Analyzer (Weeks 3-4)

Tech: Python, FastAPI, PostgreSQL, LLM API, PDF processing

Features: Resume parsing, skill extraction, JD analysis, semantic matching, scoring, suggestions

Recruiters see: a working, deployed full-stack AI product, not a notebook.

Research mentors see: correct use of embeddings for semantic similarity over naive keyword matching.

GitHub/Resume framing: Resume line: 'Built an end-to-end AI resume-JD matching system (FastAPI + PostgreSQL + LLM API) with semantic-similarity scoring, deployed with Docker.'

Interview questions it may generate: Likely interview Qs: Why embeddings over TF-IDF here? How did you handle PDF parsing edge cases? How would you evaluate matching quality without ground truth?

Project 2 — ResNet Implementation (Week 6)

Tech: PyTorch

Features: Baseline CNN, ResNet, no-residual ablation, training/validation analysis

Recruiters see: you can implement and train a real architecture, not just call `model.fit()`.

Research mentors see: correct ablation methodology — the single most research-relevant skill this project demonstrates.

GitHub/Resume framing: Resume line: 'Implemented ResNet from scratch in PyTorch and ran a controlled ablation isolating the effect of residual connections on convergence and accuracy.'

Interview questions it may generate: Likely interview Qs: What exactly is the degradation problem? Why does the ablation prove residuals help optimization specifically, not just capacity?

Project 3 — Transformer From Scratch (Week 7)

Tech: PyTorch

Features: Q/K/V, attention, multi-head attention, positional encoding, encoder/decoder

Recruiters see: you understand the architecture behind every modern LLM, not just the API.

Research mentors see: ability to translate equations into working code.

GitHub/Resume framing: Resume line: 'Implemented the Transformer architecture (multi-head self-attention, positional encoding, encoder-decoder stack) from scratch in PyTorch.'

Interview questions it may generate: Likely interview Qs: Derive scaled dot-product attention. Why do we need multiple heads instead of one large one? What breaks without positional encoding?

Project 4 — AI Research Assistant (Week 9)

Tech: Next.js, React, FastAPI, PostgreSQL, embeddings, vector DB, RAG, LLM, Docker

Features: Paper upload, semantic search, Q&A, paper comparison, methodology explanation, gap-finding

Recruiters see: a genuinely useful, portfolio-defining full-stack AI application.

Research mentors see: you understand retrieval quality is the bottleneck, and you measured it.

GitHub/Resume framing: Resume line: 'Built a RAG-based research assistant (Next.js + FastAPI + vector DB) that answers questions over uploaded papers with citation-grounded responses.'

Interview questions it may generate: Likely interview Qs: How do you choose chunk size? How do you detect when retrieval fails? Why rerank after retrieval?

Project 5 — AI + QKD Security (Week 12)

Tech: BB84 simulation, feature engineering, classical ML, PyTorch (where justified)

Features: BB84 simulation, channel/noise/attack modeling, QBER + temporal features, attack classification, ablation, robustness evaluation

Recruiters see: a rare, genuinely differentiated project at the intersection of security and AI.

Research mentors/professors see: critical literature engagement (Section 19), not just a classifier demo — this is your strongest Master's/outreach asset.

GitHub/Resume framing: Resume line: 'Designed an ML-based security-analysis pipeline for BB84 QKD, engineering temporal/channel features and comparing classical ML models against a fixed-QBER-threshold baseline across multiple attack types.'

Interview questions it may generate: Likely interview Qs: Why is session-level QBER insufficient? What's your false-negative rate and why does it matter more than accuracy here? Is the quantum component doing real work?

22 Internship Preparation

By Day 90 your portfolio targets: AI/ML internships, AI Engineer internships, GenAI internships, ML Engineer internships, AI Research internships, Research Assistant positions, and backend+AI roles.

Portfolio (in application order of strength):

- AI Research Assistant (Project 4) — leads every application; most impressive to both recruiters and research mentors.
- AI + QKD Security (Project 5) — your differentiator; nobody else applying will have this.
- Transformer From Scratch (Project 3) — proves fundamentals beyond API usage.
- ResNet Implementation (Project 2) — proves you can run a controlled experiment.
- AI Resume Analyzer (Project 1) — proves full-stack shipping ability.

For AI Engineer/GenAI roles, lead with Projects 1 and 4 (product-shaped). For AI Research/Research Assistant roles, lead with Projects 2, 3, and 5 (methodology-shaped). Tailor which two projects go at the top of your resume per role type.

23 Master's Preparation

Positioning statement to use consistently across SOPs, resumes, and outreach emails:

"E&TC Engineering → AI/ML → Research Engineering → Quantum Communication + AI" — not simply "E&TC student learning AI."

What this roadmap contributes to a Master's application:

- **Technical skills:** PyTorch, Transformers implemented from scratch, RAG systems, LoRA/QLoRA fine-tuning — demonstrable, not just listed.
- **Research experience:** one full reproduction + ablation (Week 11) and one small original study (Project 5) — the two things SOP reviewers actually look for.
- **Publication potential:** Project 5, if it finds a real gap (Section 20), is workshop-paper or preprint-shaped within a semester of follow-up work.
- **Project portfolio:** five projects spanning engineering, deep learning, and a genuine specialization — coherent enough to name in an SOP paragraph.
- **Paper-reading & reproduction ability:** demonstrable via your research notebook (Section 25).

24 Professor / Research Outreach Preparation

This roadmap supports credible outreach to professors working in AI, ML, Quantum Computing, Quantum Communication, QKD, QML, AI+Cybersecurity, AI+6G, and wireless security.

What makes an outreach email credible (vs. a generic 'I am interested in your work' email):

- Reference a specific paper of theirs, and one specific, informed critique or question about it — Section 19's critical-evaluation framework is exactly the tool for generating this.
- Attach or link Project 5 (or its interim results), since it directly overlaps with QKD/AI-security research groups' interests.
- State a specific research gap you identified (Section 20) and ask a targeted question about whether it is already being addressed in their lab — this signals you read the field, not just their homepage.
- Mention your existing BB84 final-year project as prior domain grounding, not as the whole pitch.

25 Research Notebook Template

Copy this template for every paper you deep-read or condensed-read. Keep it in one running document (Notion/Obsidian/plain markdown) organized by week.

Field	Fill in for each paper
Paper	
Date	
Research Area	
Problem	
Motivation	
Previous Approaches	
Main Contribution	
Mathematical Formulation	
Architecture	
Algorithm	
Dataset	
Experimental Setup	
Results	
Ablation	
Limitations	
Assumptions	
Reproducibility	
My Understanding	
What I Don't Understand	
Research Questions	
Possible Extensions	
Connection to My Project	
Connection to AI + Quantum	
Connection to AI + 6G	
Implementation Status	
Reproduction Status	
Final Takeaway	

26 Weekly Research Schedule

Approximately 5 hours/week on papers as a default — not a hard limit. Foundational weeks (6, 7, 9, 12) may need more; pure-engineering weeks (1, 3) may need less or none.

Activity	Typical time	Notes
Paper reading (Passes 1-3)	~2 hrs	Spread across 2-3 sessions, not one sitting
Mathematical preparation	~1 hr	Only in weeks introducing new math (2, 5, 7)
Coding / implementation (Pass 4)	Project-time, tracked separately	This is the bulk of weekly hours, not paper time
Notes (research notebook)	~30-45 min	Fill in immediately after Pass 3, before implementing
Experiment / ablation	Varies by week	Concentrated in Weeks 6, 7, 11, 12
Research reflection (Pass 5)	~30 min	End of week — write 1-2 research questions down

27 90-Day Checklist

Engineering

- Python fluency + Git/GitHub workflow
- NumPy/Pandas/Matplotlib comfort
- FastAPI backend with auth + validation
- PostgreSQL schema design + SQL joins/indexes
- React/Next.js frontend shipped
- Docker + deployment of at least 2 projects

AI / ML

- Classical ML models trained + correctly evaluated
- Backpropagation derived and explained without notes
- CNN + ResNet implemented, with a completed ablation
- Transformer implemented from scratch
- BERT/GPT fine-tuned on a downstream task
- RAG pipeline built end-to-end with a vector database
- LoRA/QLoRA fine-tuning run at least once

Research

- Research notebook completed for every Tier 1 paper
- One full reproduction (with ablation and discrepancy analysis) completed
- Critical-evaluation framework applied to at least 3 AI+QKD papers
- At least one original research question and hypothesis written down

Projects

- Project 1 — AI Resume Analyzer deployed
- Project 2 — ResNet + ablation documented
- Project 3 — Transformer from scratch working
- Project 4 — AI Research Assistant deployed
- Project 5 — AI + QKD Security study completed and written up

28 Papers I Should Not Read Yet

Paper / Area	Why deferred
Full PPO/RLHF derivations (beyond InstructGPT's conceptual level)	Requires RL background not built in this 90 days; DPO gives most of the practical benefit at a fraction of the prerequisite cost.
GPT-4/o1-class closed technical reports	Sparse on technical detail by design — good for capability awareness, not for implementable learning; revisit for context only, never as a deep-read.
Full quantum-computing textbook derivations (e.g. quantum error correction, Shor's algorithm proof)	Not required for your AI+QKD security-analysis direction, which is classical ML applied to quantum-protocol data, not quantum-algorithm design. Revisit only if you pivot toward QML circuit design specifically.
Adversarial Reinforcement Learning for Adaptive Eavesdropping (2026)	Frontier/adaptive-attacker framing requires RL fundamentals this roadmap does not cover; skim now (done in Section 19), deep-read only if this becomes your thesis direction.
Highly specialized satellite-QKD orbital-mechanics papers	Too far from your current E&TC;/BB84 base without the orbital-mechanics prerequisites; low relevance to a 90-day plan.
Agent-framework papers (ReAct, AutoGPT-style, multi-agent orchestration)	Builds on RAG/tool-use, which you only reach in Week 9-10; reading these earlier adds vocabulary without the foundation to evaluate the claims critically.
Redundant embedding-model papers (Sentence-BERT variants, later contrastive-learning papers)	Word2Vec + BERT + DPR already establish the conceptual lineage you need; additional variants are engineering choices (which embedding model to call), not new research ideas.

29 Next Steps After Day 90

- **Deepen Project 5 into a real study:** expand the attack-type coverage, add a second dataset or simulator configuration, and write it up in paper format (even if only for your GitHub/portfolio at first).
- **Reinforcement learning foundations:** now unlocks the Tier-4 adaptive-attacker papers deferred in Section 28, and modern LLM agent/tool-use research.
- **Formal quantum computing coursework** (e.g. a structured QC course) if you want to move from 'AI applied to QKD data' toward genuine QML circuit-level research.
- **Outreach:** send 3-5 targeted emails (Section 24) to professors whose recent papers you can specifically critique, using Project 5 as the credibility anchor.
- **Publication attempt:** if Project 5 surfaces a genuine, literature-supported research gap (Section 20), scope a small, focused study around it and target a workshop or a preprint before a full venue submission.
- **Keep the research notebook running** — it is the single artifact that will make your next 90 days faster than this one.

"Optimize for depth of understanding, not for the number of papers finished."